svmodt: An R Package for Linear SVM-Based Oblique Decision Trees

by Aneesh Agarwal, Jack Jewson, and Erik Sverdrup

Abstract Decision trees are widely used for classification tasks due to their representational power and interpretability, but traditional trees with axis-aligned splits often need many splits in order to approximate complex decision boundaries. Oblique decision trees split according to a linear combination of variables, providing greater flexibility, but deciding how and where to split becomes challenging. We consider linear Support Vector Machines (SVM) as a principled and computationally convenient method to learn node specified oblique splits. We provide STreeR, the first fully native R implementation of the STree algorithm implementing SVM-based oblique decision trees that use linear Support Vector Machines at each node to create multivariate splits, and validate it against the original Python implementation. We then introduce svmodt, an R package extending STree to support dynamic feature selection strategies, node-specific class weighting for handling imbalanced data, and feature diversity mechanisms through penalisation. We demonstrate that SVMODT outperforms axis-parallel decision trees while also maintaining more interpretable tree structures. The package includes detailed documentation, and is available on GitHub.

1 Introduction

Decision trees remain widely used in machine learning because they are interpretable, straightforward to apply, and can accommodate both categorical and numerical predictors with minimal preprocessing. Classical algorithms such as Classification and Regression Trees (CART, (Breiman et al., 1984)) and C4.5 (Quinlan, 2014) employ axis-aligned (univariate) splits that partition the feature space along coordinate axes, thereby preserving transparency in the resulting decision rules. Consider a supervised classification task where we wish to predict $y \in \{1, \ldots, K\}$, where K is the number of classes, from d-predictor variables $x \in \mathcal{X} \subseteq \mathbb{R}^d$. In this setting an axis parallel split constitutes selecting one feature $j \in \{1, \ldots, d\}$ and a splitting value $\theta \in \mathbb{R}$ and splits according to the test $x_j \leq \theta$.

In R, traditional decision trees are readily implemented via packages such as **rpart**, **tree**, and **party**. However, the simple split structure of axis-parallel decision trees can promote overfitting, producing duplicated subtrees and repeated evaluations of the same predictor, which reduces efficiency and weakens generalisation (Pagallo and Haussler, 1990). Oblique decision trees mitigate these limitations by permitting splits on linear combinations of features; for example, a single oblique split of the form $\sum_{j=1}^{d} w_j x_j \leq \theta$, where $w = (w_1, \ldots, w_d) \in \mathbb{R}^d$, can often replace multiple axis-aligned dplits, producing more compact and geometrically simpler partitions of the feature space. Oblique trees are available in R through packages such as **ODRF**, **aorsf**, and **oblique.tree**. Empirical studies across diverse data sets show that oblique decision trees often achieve higher accuracy and yield more compact models than axis-parallel trees, though with a modest reduction in interpretability (Murthy et al., 1994; Cañete et al., 2021).

Figure 1 contrasts an axis-parallel decision tree with an axis-oblique decision tree. The axis-parallel tree (left panel) fits an overly complex boundary, particular between the orange (circles) and purple (triangle) classes, and still has a high rate of training misclassification, reflecting its inability to capture interactions among predictors that determine class membership. In contrast, the oblique tree (right

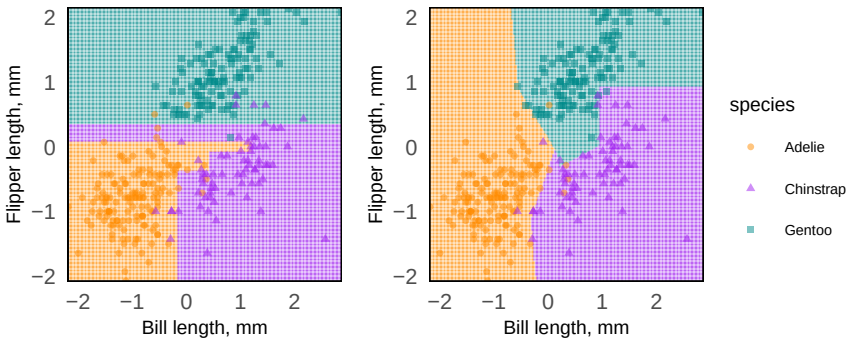


Figure 1: Comparison of axis-parallel and axis-oblique decision boundaries on the Palmerpenguins dataset using bill length (mm) and flipper length (mm) as predictor variables.

panel) yields geometrically simpler decision boundaries and substantially fewer misclassifications on the training set, indicating a better fit to the underlying multivariate relationships.

A challenge with oblique decision trees, however, is deciding upon suitable oblique splits. The simplicity of axis-parallel trees allows algorithms such as CART (Breiman et al., 1984) to consider all possible splits, as for each j there is a maximum of $n-1$ values of θ that would result in distinct splits. On the other hand, the added complexity of oblique increases the number of possible splits enormously, preventing exhaustive searches. Existing R packages address this by different means: **ODRF** uses random subspace projections, **aorsf** applies accelerated coordinate descent on the log-rank gradient, and **oblique.tree** employs linear discriminant functions as node classifiers.

Support Vector Machines (SVMs), introduced by Cortes and Vapnik (1995), provide a principled way to obtain oblique splits by finding margin-maximizing hyperplanes that separate the data in each node of the tree. This margin-based approach enhances the generalisation ability of the model, making SVMs robust and effective for many real-world problems (Cristianini, 2000), and allows for efficient estimation via quadratic programming. The idea of fitting SVMs at tree nodes was formalized by Bennett and Blue (1998) and extended to multiclass settings and other refinements in subsequent work (Takahashi and Abe, 2002; Bala and Agrawal, 2011; Ganaie et al., 2022). Python's STree implements SVM-node oblique trees using scikit-learn's SVC (Montañana et al., 2021; Pedregosa et al., 2011), while Java tools such as Weka offer components that can be adapted for hierarchical SVM trees (Menkovski et al., 2008). However, fully featured, native R implementations does not exist.

We introduce STreeR, a fully native R implementation of an STree-style support vector machine oblique decision tree that operates independently of Python. This implementation provides a production-ready interface and leverages vectorised computation to ensure computational efficiency and seamless integration within the R ecosystem. Second, we develop SVMODT, an enhanced support vector machine-based oblique decision tree that extends the original STree framework through several practical innovations. These include feature subsetting, which enables the selection of only a subset of available predictors at each node. While later versions of STree introduced basic feature subsetting, SVMODT generalises this idea through additional strategies, including decreasing and random feature subsetting schemes that adapt the candidate feature space throughout the depth of the tree. SVMODT also incorporates embedded feature selection, allowing predictors to be prioritised using mutual information, correlation-based association measures, or randomised ranking procedures. In addition, SVMODT introduces a feature penalisation mechanism that discourages repeated reuse of the same predictors across successive splits, thereby promoting greater feature diversity throughout the tree structure. The framework further extends STree by incorporating explicit pruning and node-specific class weighting schemes, neither of which are available in the original STree implementation.

2 Background

Decision Trees

Decision Trees (DTs) are interpretable classification models that represent their decision-making process through a hierarchical, tree-like structure. This structure comprises internal nodes containing splitting criteria that divide up the predictor space and terminal (leaf) nodes corresponding to predicted outcomes (class labels and probabilities). The nodes are connected by directed edges, each representing a possible True or False outcome of a splitting criterion for observations whose feature reach this node.

Decision tree algorithms can be categorized based on whether the same type of split-test is applied at all internal nodes. **Homogeneous trees** employ a single algorithm throughout (e.g., univariate or multivariate splits), whereas **hybrid trees** allow different algorithms such as linear discriminant functions, k-nearest neighbors, or univariate splits that can be used in different sub-trees (Brodley and Utgoff, 1995). Hybrid trees exploit the principle of *selective superiority*, allowing subsets of the data to be modeled by the most appropriate classifier, thereby improving flexibility and accuracy. However, hybrid tree models typically incur substantial computational overhead due to their structural complexity, and therefore, in this paper we restrict our focus to homogeneous trees.

Axis-Parallel Decision Trees

Axis-Parallel Decision trees divide the feature space into several disjoint axis-parallel hypercubes . Axis-parallel decision trees, such as CART and C4.5, represent two of the most widely used algorithms for classification tasks. The CART algorithm employs a binary recursive partitioning procedure that is capable of handling both continuous and categorical variables as predictors or targets. At each node, the algorithm systematically evaluates every available variable $j \in \{1, \ldots, d\}$ and all of its potential split points θ i.e. between any two observations, to determine the optimal partition.

In contrast, **C4.5**, an extension of the earlier **ID3** algorithm (Quinlan, 1986), utilizing information theory measures such as **information gain** and **gain ratio** to select the most informative attribute for each split (Quinlan, 2014). C4.5 also includes mechanisms to handle missing attribute values by weighting instances according to the proportion of known data and employs an **error-based pruning** to eliminate subtrees that fail to improve predictive performance beyond a threshold (α), thereby mitigating overfitting (Mingers, 1989; Schaffer, 1992).

Axis-parallel decision trees offer strong interpretability but face inherent representational limits. Their constrained split structure often necessitates deep trees which can induces overfitting, leading to duplicated subtrees and repeated tests of the same predictor, ultimately reducing efficiency and weakening generalisation (Pagallo and Haussler, 1990).

Axis-Oblique Decision Trees

Axis-oblique decision trees extends axis-parallel decision trees by allowing each internal node to perform splits based on linear or nonlinear combinations (Chabbouh et al., 2025) of multiple features. This flexibility enables the tree to form oblique decision boundaries that more accurately partition the instance space. For example, a single multivariate test such as $x + y < 8$ can replace multiple univariate splits needed to approximate the same boundary.

The construction of Axis-oblique decision trees introduces several design considerations, including how to represent multivariate tests, determine their coefficients, select features to include, handle symbolic and missing data, and prune to avoid over-fitting (Brodley and Utgoff, 1995). Various optimisation algorithms such as recursive least squares (Young, 1984), the pocket algorithm (Gallant, 1986), or thermal training (Frean, 1990) may be used to estimate the weights. However , Axis-oblique decision trees trade interpretability for representational power and often require additional mechanisms for **local feature selection**, such as *sequential forward selection* (SFS) or *sequential backward elimination* (SBE) (Kittler, 1986).

Although oblique splits introduce additional complexity and reduce interpretability, oblique decision trees retain key advantages of standard decision trees such as sequential split criteria evaluation and transparent decision procedures while offering improved modeling flexibility for complex data sets (Koziol and Wozniak, 2009; Friedl and Brodley, 1997; Liu and Setiono, 1998).

Alternate Approaches to Oblique Decision Boundaries

Employing linear classifiers at internal nodes can yield decision trees that are both accurate and diverse. Because each node receives a different subset of the training data, the most suitable classification boundary may vary from node to node. For example, (Menze et al., 2011; Lemmond et al., 2010) use Linear Discriminant Analysis (LDA) to construct linear splits, though their approach is limited to binary classification. In contrast, Truong (2009) introduced multiple logistic-regression–based partitions (specifically $2^{K-1} - 1$ for K classes) to select selects the split that best optimizes the impurity criterion.

Projection pursuit methods (Friedman and Tukey, 2006; Kruskal, 1969) have also been used to identify low-dimensional projections that expose structure in high-dimensional data. The **PPtree**(Lee et al., 2013; da Silva et al., 2021) algorithm extends this idea into a recursive partitioning framework by optimizing a class-sensitive projection index at each node and then applying standard split rules to the resulting one-dimensional projection. Similarly, Bulò and Kontschieder (2014) introduce a randomized multi-layer perceptron as a node-level splitting function. However, determining an appropriate network complexity remains challenging, as overly flexible models risk substantial overfitting.

Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are supervised learning models used predominatly for binary classification tasks, although extensions to multi-class classification and regression do exist. They aim to determine the optimal separating hyperplane that maximizes the margin between the two classes. This margin-based approach enhances the generalisation ability of the model, making SVMs robust and effective for many real-world problems (Cristianini, 2000).

Linear SVMs construct a separating hyperplane $f(x) = \text{sign}(w^\top x + b)$ in the d-dimensional feature space such that the distance between the closest points of each class (the **support vectors**) and the hyperplane itself is maximised (Cortes and Vapnik, 1995). Given a training dataset $\{(x_i, y_i)\}_{i=1}^N$, where $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$, training an SVM (estimating w) entails solving a convex quadratic programming (QP) problem involving an $(n \times n)$ kernel matrix ($K_{jk} = k(x_j, x_k) = x_j x_k^\top$). Owing to the convexity of the objective, the QP admits a unique global optimum and can be solved reliably and

efficiently using standard optimisation techniques. While linear classifiers provide useful insights, they are often inadequate for real-world data sets, where classes are not linearly separable. In such cases, SVMs can be extended to create **nonlinear decision boundaries** by mapping the input vectors into a higher-dimensional **feature space** using a nonlinear transformation $\phi : \mathbb{R}^d \to \mathcal{F}$. So doing is facilitated computationally by the kernel trick which simplifies $K_{jk} = k(x_j, x_k) = \phi(x_j)\phi(x_k)^\top$.

Although SVMs exhibit strong theoretical foundations and robust generalisation capabilities, they present several practical limitations, most notably their lack of interpretability and effective methods for feature selection. Further, model performance is highly dependent on the appropriate selection of hyperparameters such as the regularisation term (C) and kernel parameters (e.g., γ), which govern the trade-off between margin maximization and misclassification tolerance (Nanda et al., 2018). For large-scale data sets, training an SVM can become computationally expensive, with both runtime and memory consumption growing quadratically in the number of training samples (Dong et al., 2005). Moreover, SVMs are inherently designed for binary classification, necessitating decomposition strategies such as One-vs-One and One-vs-All for multi-class problems (Hsu and Lin, 2002). Their performance also tends to degrade in imbalanced data settings, where the decision boundary becomes biased toward the majority class (Cervantes et al., 2020).

Oblique Decision Trees with Support Vector Machines

Support Vector Machine–based decision trees were first formalized by Bennett and Blue (1998), who adapted Statistical Learning Theory and Structural Risk Minimization to construct binary decision trees in which each internal node is an SVM that partitions the data by an optimal hyperplane. This formulation enabled multivariate, margin-based decisions at every split. Takahashi and Abe (2002) extended the idea to multiclass settings by using a recursive partitioning strategy: the root node isolates the most separable class or classes from the remainder, and the procedure recurses until each leaf contains a single class, thereby covering all classes. Subsequent studies have built upon this foundation to address scalability, optimisation, and generalisation issues. Optimal Decision Tree SVM (ODT-SVM) (Bala and Agrawal, 2011) introduced split-selection criteria based on Gini index, information gain, and scatter matrix separability to balance tree interpretability and margin-based precision. Reynolds et al. (2019) further extended the framework by incorporating class-specific cost parameters through the assignment of weights to individual classes, improving performance in imbalanced classification settings.

STree

More recent approaches embed multiple SVM classifiers directly within each split to handle multi-class problems more efficiently. STree (Montañana et al., 2021; Montañana et al., 2021) is an oblique multi-class decision tree algorithm that integrates support vector machines (SVMs) to construct single margin-based splits capable of handling multiclass problems. Unlike traditional multi-class tree methods that rely on clustering or ensembles of binary models, STree builds a single decision tree. Its central innovation lies in using SVM-derived hyperplanes at each internal node, enabling more expressive splits than axis-aligned trees while also being interpretable and computationally simple.

The algorithm generates a binary tree recursively (see Algorithm 5 in the Appendix). At each node:

1) Stopping conditions are evaluated to check the depth of the tree and the class purity. If the stopping conditions are met, a leaf node is created, labelled with the most frequent class in the node.
2) If the stopping conditions haven't been met the algorithm then selects the best hyperplane to split the data into two partitions: T^+ (positive side) and T^- (negative side).

When all instances at a node belong to more than two classes ($k' > 2$), the approach generates multiple candidate binary splits: For each class label y_i, it constructs a one-vs-rest binary problem: Class y_i vs. all other classes. It trains an SVM for each of these k' problems, resulting in k' hyperplanes H_i, $i = 1, \ldots, K$. Each hyperplane partitions the data into T_i^+ and T_i^-. The algorithm computes the impurity (using Shannon entropy) of the class distribution within each partition. When only two classes remain, STree trains a single SVM to obtain the maximum-margin hyperplane. Instances are then routed left or right based on the sign of their distance to this hyperplane. Stree's performance depends heavily on hyperparameter tuning, including kernel choice (linear, polynomial or Gaussian), gamma, polynomial degree, regularisation parameter C, and max iteration.

Table 1: Comparison of mean prediction accuracy and median training time for STreeR and STree. Here, N denotes the number of observations, X the number of features, and L the number of classes.

Dataset	N	X	L	STreeR	STree	STreeR(Med)	Stree(Med)
WDBC Diagnosis	569	30	2	**0.970 ± 0.004**	**0.970 ± 0.003**	0.043ms	0.019ms
Iris	150	4	3	**0.966 ± 0.006**	0.965 ± 0.010	0.011ms	0.007ms
Echocardiogram	131	10	2	**0.846 ± 0.004**	0.845 ± 0.008	0.007ms	0.006ms
Fertility	100	9	2	**0.876 ± 0.011**	0.874 ± 0.010	0.003ms	0.005ms
Wine	178	13	3	**0.978 ± 0.008**	0.959 ± 0.008	0.023ms	0.007ms
Cardiotocography-3	2126	21	3	0.901 ± 0.003	**0.903 ± 0.003**	0.97ms	0.242ms
Cardiotocography-10	2126	21	10	0.792 ± 0.004	**0.795 ± 0.018**	4.391ms	0.474ms
Ionosphere	351	33	2	**0.896 ± 0.009**	0.892 ± 0.015	0.049ms	0.021ms
Dermatology	366	34	6	**0.972 ± 0.006**	0.959 ± 0.004	0.126ms	0.019ms

3 Contributed Methods

STreeR

We introduce STreeR, a R implementation of the STree algorithm. As no such implementation currently exists, we reconstruct the method in R using the **e1071** package, which interfaces with the LIBSVM C++ backend. This reconstruction provides greater transparency into the algorithm's internal workflow and enables fair benchmarking against competing approaches. Algorithm 1 provides a concise summary of the recursive splitting underlying the STreeR implementation.

Algorithm 1: STreeR: Multi-class OVR SVM-based oblique decision tree

Input: $\mathcal{D}' = \{(x_i, y_i)\}_{i=1}^{l'}$: Training Data
Output: *tree*: leaf node or data split $\mathcal{D}^{'+}, \mathcal{D}^{'-}$

function STree($\mathcal{D}'$):
  if $stopping\_condition(\{y_i\}_{i=1}^{t})$ **then**
    return $create\_leaf\_node(mode(\{y_i\}_{i=1}^{t}))$ // Leaf node
  $k' \leftarrow num\_different\_labels(\{y_i\}_{i=1}^{t})$
    $\mathcal{Y}' \leftarrow \{y'_1, \ldots, y'_{k'}\}$ // set of labels
  $I_{all} \leftarrow I(\{y_i\}_{i=1}^{t})$ // $I(\cdot)$ is an information theory measure
  $n_{models} \leftarrow k'$ // OvR
  for $j = 1$ **to** n_{models} **do**
    if $(j = k' = 2)$ **then**
      break // Two labels, one SVM is enough
    $models[j] \leftarrow \text{SVM}(\mathcal{D}')$ using binary class y'_j (+) vs rest (–) // OvR
  $\mathcal{D}^{'+}, \mathcal{D}^{'-} \leftarrow models[j](\overrightarrow{x}_j) \quad \forall(\overrightarrow{x}) \in \mathcal{D}'$
  $I_j \leftarrow \frac{|\mathcal{D}^{'+}|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}^{'+}|}) + \frac{|\mathcal{D}^{'-}|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}^{'-}|})$
  $IG_j \leftarrow I_{all} - I_j$
  $b^* = \arg\max_{j=1,\ldots,n_{models}} IG_j$
  if $IG_{b^*} > 0$ **then**
    $node \leftarrow create\_node(models[b^*])$
    $node.left \leftarrow STree(\mathcal{D}^{'+})$
    $node.right \leftarrow STree(\mathcal{D}^{'-})$
  else
    return $create\_leaf\_node(model(\{y_i\}_{i=1}^{t}))$ // No split gain
  return *node*

To verify the accuracy of our R implementation of STree, we conducted a benchmarking comparison against the original Python version using a 5-fold cross-validation repeated 10 times with different seeds applied to 9 benchmark datasets from the UCI Machine Learning Repository. For comparability, both algorithms were run with the same default hyperparameters: cost-complexity $C = 1$, a linear kernel, a maximum of 1 x 10^7 training iterations, and a maximum depth of 10. Prediction accuracies were averaged across all folds for each dataset.

Across most datasets, STreeR exhibits performance that is effectively identical to its Python counterpart (see Table 1) . Although both implementations rely on the same underlying `LIBSVM` C++ library, Python through the `SVC` classifier and R through the `e1071::svm` interface, there exist subtle yet meaningful differences in how each framework invokes the underlying routines responsible for fitting the support vector machine decision boundary and applying scaling transformations during prediction. These implementation-level discrepancies are likely to contribute to the small observed variation in performance. To mitigate differences attributable to feature scaling, each dataset has been pre-processed in R using an implementation that replicates the behaviour of Python's standard scaling procedure.

We restrict our implementation of STree to the one-vs-rest (OVR) splitting strategy. The alternative one-vs-one scheme is not included, as it scales poorly with the number of classes: for problems with large L, the number of pairwise SVMs grows quadratically, substantially increasing training time without offering clear benefits for the aspects of the algorithm we aim to study. We also implement only a linear kernel, both to maintain the interpretability of the resulting tree structure and to avoid the considerable computational overhead associated with non-linear kernels.

Support Vector Machine based Oblique Decision Trees: SVMODT

Support Vector Machine-based Oblique Decision Tree (SVMODT) extends the STree framework to incorporate feature subsetting, dynamic feature selection, pruning, feature penalisation, and class weighting.While several components, such as dynamic feature subsetting, are reconstructed from the original STree framework, we also refine existing mechanisms including feature subsetting and pruning to provide greater adaptability. We also introduce new controls for feature penalisation and node-specific class weighting. We further simplify the original STree structure by restricting node classifiers to linear kernels, thereby removing the need for multiple kernel-specific hyperparameters and improving interpretability, computational efficiency, and ease of inference.

At each node, the algorithm (see Algorithm 4), implemented in R using the **e1071** package interface to LIBSVM, performs the following operations. A subset of features is first selected according to a specified strategy, which may remain constant, decrease with tree depth, or be randomly sampled within a predefined range. Within this subset, features are then prioritised using a criterion such as mutual information, correlation, or random assignment. To encourage diversity across splits, a penalisation mechanism reduces the likelihood of repeatedly selecting features that have been used in previous nodes.

Given the selected feature set, a binary support vector machine is trained at node v, optionally incorporating class-specific weights to account for imbalance in the local data distribution. The fitted classifier defines a separating hyperplane characterised by a weight vector w_v^*, and observations are partitioned according to the sign of the decision function $\text{sign}((w_v^*)^\top x)$. The resulting split is evaluated using an impurity-based criterion, and the procedure is recursively applied to the resulting child nodes until a stopping condition is satisfied. We now discuss the key features of SVMODT and how these differentiate it from the original STree framework.

Key Features

Binary & Multiclass SVM Training At each internal node of the tree, a Support Vector Machine (SVM) is trained to construct an oblique decision boundary that partitions the data into two child nodes. The training procedure differs depending on whether the node contains samples from two classes or more than two classes. The binary and multiclass split follow the same procedure as Stree using one vs rest SVM splits except they have been slightly modified to incorporate class-weighting and pruning strategy.

Binary Classification When the node contains exactly two classes ($k = 2$), a binary SVM classifier is trained (see Algorithm 2) using the selected feature subset $\mathcal{F}_d$. Let $\mathbf{X}_{\text{scaled}}$ denote the standardized feature matrix at that node and $\mathcal{Y}$ the corresponding class labels.

Algorithm 2: Binary Classification Split

Input: $\mathbf{X}_{\text{scaled}}$: scaled features, $\mathcal{Y}$: labels, strat_w: class weighting strategy
Output: $(\text{model}, \mathcal{I}_L, \mathcal{I}_R)$: SVM model, left indices, right indices, split class

function `BinarySplit`$(\mathbf{X}_{scaled}, \mathcal{Y}, strat_w)$**:**
  $w_c \leftarrow \text{calculate\_class\_weights}(\mathcal{Y}, \text{strat}_w)$
  $\text{model} \leftarrow \text{fit\_linear\_SVM}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}, w_c)$
  if *model* $=$ *null* **then**
    return $(\text{null}, \emptyset, \emptyset, \text{null})$
  $\mathbf{f} \leftarrow \text{model.decision\_values}(\mathbf{X}_{\text{scaled}})$
  $\mathcal{I}_L \leftarrow \{i : f_i > 0\}, \quad \mathcal{I}_R \leftarrow \{i : f_i \leq 0\}$
  return $(\text{model}, \mathcal{I}_L, \mathcal{I}_R, \text{null})$

The SVM learns a separating hyperplane of the form:

$$\begin{aligned} \underset{\boldsymbol{\beta}\in\mathbb{R}^{d+1}, \boldsymbol{\xi}\in\mathbb{R}^N_+}{\arg\min} \quad & \frac{1}{2}\boldsymbol{\beta}_{1:d}^\top \boldsymbol{\beta}_{1:d} + C\sum_{n=1}^{N}\xi_n \\ \text{subject to} \quad & y_n\left(\boldsymbol{\beta}^\top \mathbf{x}_n\right) \geq 1-\xi_n, \qquad n=1,\ldots,N, \\ & \xi_n \geq 0, \qquad n=1,\ldots,N. \end{aligned}$$

where β represents the weight vector and ξ_n are slack variables that permit margin violations. The hyperplane partitions the data into two subsets according to the value of the decision function. Samples satisfying $f(x) \leq 0$ are assigned to the left child node, while those with $f(x) > 0$ are assigned to the right child node. Class weights may optionally be applied during SVM training to address class imbalance. These weights influence the optimisation process by increasing or decreasing the penalty associated with misclassification of observations from particular classes. The resulting partition indices $\mathcal{I}_L$ and $\mathcal{I}_R$ are then used to recursively construct the left and right subtrees.

Multiclass Classification When more than two classes are present at the node ($k > 2$), a multiclass SVM strategy is employed (see Algorithm 3) to generate a binary split. The algorithm evaluates candidate partitions of the class set in order to determine a split that maximizes impurity reduction. For each candidate split, the classes are divided into two groups, and a binary SVM is trained to separate the corresponding subsets of samples. The quality of the resulting partition is evaluated using an impurity reduction criterion such as information gain. Let $I(\mathcal{Y}_P)$ denote the impurity of the parent node and $I(\mathcal{Y}_L)$ and $I(\mathcal{Y}_R)$ denote the impurities of the resulting child nodes. The information gain is computed as

$$\Delta I = I(\mathcal{Y}_P) - \left(\frac{|\mathcal{I}_L|}{n_P} I(\mathcal{Y}_L) + \frac{|\mathcal{I}_R|}{n_P} I(\mathcal{Y}_R)\right),$$

where n_P is the number of samples at the node. Candidate splits that produce an information gain below the minimum threshold τ_I are discarded. Similarly, splits resulting in empty child nodes are considered invalid and excluded from consideration. SVM fitting may also fail when the binary target contains only a single class, when one class contains too few observations, or when numerical optimisation fails to converge. Such candidate models are skipped. Among the remaining valid candidates, the split with the lowest weighted post-split impurity (equivalently, the highest information gain) is selected. If no valid candidate split is found, the node is converted into a terminal leaf node.

Algorithm 3: Multiclass One-vs-Rest Split

Input: $\mathbf{X}_{\text{scaled}}$: scaled features, $\mathcal{Y}$: labels, strat_w: class weighting strategy, I_{parent}: parent impurity, measure_I: impurity measure, τ_I: minimum information gain, n: number of samples

Output: (best_model, $\mathcal{I}_L, \mathcal{I}_R$): best SVM model, left indices, right indices, split class

function `MulticlassSplit`($\mathbf{X}_{scaled}, \mathcal{Y}, strat_w, I_{parent}, measure_I, \tau_I, n$)**:**

- $k \leftarrow |\text{unique}(\mathcal{Y})|$
- $n_{\text{models}} \leftarrow k$
- **for** $j = 1$ **to** n_{models} **do**
  - $c_j \leftarrow \text{unique}(\mathcal{Y})[j]$ `// Target class for OvR`
  - $\mathcal{Y}_{\text{binary}} \leftarrow \begin{cases} + & \text{if } y_i = c_j \\ - & \text{otherwise} \end{cases}$
  - $w_c \leftarrow \text{calculate\_class\_weights}(\mathcal{Y}_{\text{binary}}, \text{strat}_w)$
  - $\text{model}_j \leftarrow \text{fit\_linear\_SVM}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}_{\text{binary}}, w_c)$
  - **if** $|\mathcal{I}_L^j| = 0$ ***or*** $|\mathcal{I}_R^j| = 0$ **then**
    - **continue**
  - $\mathbf{f}_j \leftarrow \text{model}_j.\text{decision\_values}(\mathbf{X}_{\text{scaled}})$
  - $\mathcal{I}_L^j \leftarrow \{i : f_{j,i} > 0\}, \quad \mathcal{I}_R^j \leftarrow \{i : f_{j,i} \leq 0\}$
  - **if** $|\mathcal{I}_L^j| = 0$ ***or*** $|\mathcal{I}_R^j| = 0$ **then**
    - **continue** `// Skip degenerate split`
  - $I_L^j \leftarrow I(\mathcal{Y}[\mathcal{I}_L^j], \text{measure}_I)$
  - $I_R^j \leftarrow I(\mathcal{Y}[\mathcal{I}_R^j], \text{measure}_I)$
  - $I_j \leftarrow \frac{|\mathcal{I}_L^j|}{n} \cdot I_L^j + \frac{|\mathcal{I}_R^j|}{n} \cdot I_R^j$
  - $IG_j \leftarrow I_{\text{parent}} - I_j$
  - **if** $IG_j < \tau_I$ **then**
    - **continue** `// Insufficient information gain`
- $b^* \leftarrow \arg\min_j I_j$ `// Select best split`
- **if** $model_{b^*} = null$ **then**
  - **return** $(\text{null}, \emptyset, \emptyset, \text{null})$
- **return** $(\text{model}_{b^*}, \mathcal{I}_L^{b^*}, \mathcal{I}_R^{b^*})$

Maximum Feature Selection To control model complexity and encourage feature diversity across tree nodes, the algorithm dynamically limits the number of candidate features considered during split generation. Let p denote the total number of available predictors. At depth d of the tree, the maximum number of features considered is denoted by m_d. The value of m_d is determined using one of three configurable strategies:

$$m_d = \begin{cases} m_{\text{base}} & \text{constant strategy} \\ \lfloor m_{\text{base}} \cdot \alpha^{d-1} \rfloor & \text{decreasing strategy} \\ \text{Uniform}(\lfloor p \cdot \ell_{\min} \rfloor, \lfloor p \cdot \ell_{\max} \rfloor) & \text{random strategy} \end{cases}$$

In the **constant strategy**, the same number of candidate features m_{base} is evaluated at every depth of the tree. This provides consistent computational cost across nodes.

In the **decreasing strategy**, the number of candidate features gradually decreases as the tree depth increases. The parameter $\alpha \in (0, 1]$ controls the rate of reduction. This approach allows early splits to explore a larger feature space while later splits focus on a smaller set of predictors.

In the **random strategy**, the number of candidate features is sampled from a uniform distribution bounded by fractions $\ell_{\min}$ and $\ell_{\max}$ of the total feature count p. This introduces additional stochasticity into the tree construction process and may improve generalisation by reducing overfitting.

Once m_d is determined, up to m_d features are selected using the specified feature selection method. The resulting subset is then used to train the SVM model responsible for generating the oblique split at the node.

Figure 2 displays the decision surface of the fitted SVMODT model under alternative specifications of the maximum feature selection strategy (see the HTML version for the multidimensional `tourr`-based visualisation). The coloured regions represent the predicted class labels of the fitted tree

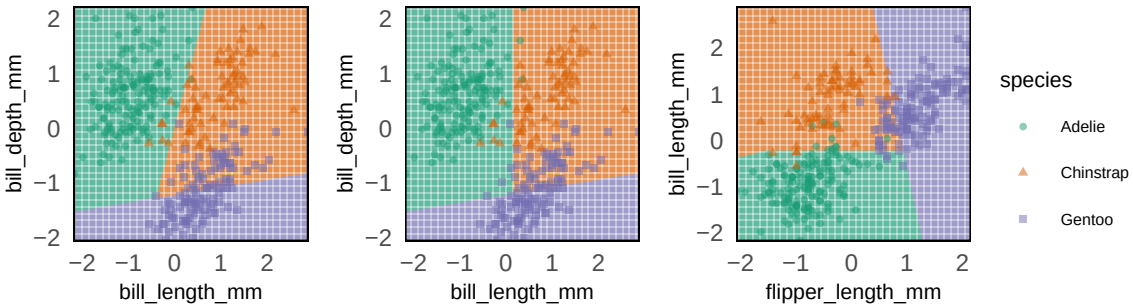


Figure 2: Comparison of constant, decreasing, and random maximum feature selection strategies using SVMODT on the Palmer Penguins dataset.

Table 2: Comparison of mean prediction accuracy (mean ± standard deviation) across maximum feature selection strategies for SVMODT, with results benchmarked against the Python STree implementation All SVMODT models were trained using maximum depth parameter ($d_{\max} = 15$), mutual-information feature ranking, and adaptive feature selection. The constant strategy uses a fixed number of candidate features at each node; the decrease strategy progressively reduces the number of available features with tree depth ($\alpha = 0.5$); the random strategy samples the candidate feature proportion uniformly from the interval $l_{\min} = 0.5$ to $l_{\max} = 0.8$ at each split. Bold values indicate the highest prediction accuracy for each dataset.

Dataset	N	X	L	STree	Constant	Decrease	Random
WDBC Diagnosis	569	30	2	**0.970 ± 0.003**	0.969 ± 0.003	**0.970 ± 0.004**	0.967 ± 0.008
Iris	150	4	3	0.965 ± 0.010	**0.966 ± 0.006**	0.955 ± 0.003	0.963 ± 0.006
Echocardiogram	131	10	2	0.845 ± 0.008	0.846 ± 0.006	0.846 ± 0.004	**0.850 ± 0.005**
Fertility	100	9	2	0.874 ± 0.010	0.874 ± 0.010	0.875 ± 0.011	**0.881 ± 0.000**
Wine	178	13	3	0.959 ± 0.008	**0.978 ± 0.008**	0.976 ± 0.006	0.959 ± 0.009
Cardiotocography-3	2126	21	3	0.903 ± 0.003	0.902 ± 0.004	0.890 ± 0.002	**0.906 ± 0.005**
Cardiotocography-10	2126	21	10	0.795 ± 0.018	0.792 ± 0.004	0.630 ± 0.007	**0.797 ± 0.006**
Ionosphere	351	33	2	0.892 ± 0.015	**0.896 ± 0.011**	0.892 ± 0.012	0.884 ± 0.013
Dermatology	366	34	6	0.959 ± 0.004	0.972 ± 0.006	**0.979 ± 0.006**	0.971 ± 0.005

evaluated over a grid of values, with the observed data points overlaid for reference. The first two selected features define the horizontal and vertical axes, while all remaining features are fixed at their median values.

The figure illustrates how the maximum feature selection strategy influences the geometry of the induced decision boundaries. Under the constant strategy, the model retains the same number of candidate variables at each node, resulting in more pronounced oblique splits across the two plotted dimensions. Under the decreasing strategy, the number of available features declines with depth, and the separation between Adelie and Chinstrap becomes increasingly axis-parallel. Information loss in deeper nodes restricts the model's ability to construct fully oblique partitions. Under the random strategy, the subset of candidate variables varies stochastically across nodes, producing some variation in the selected predictors. Nevertheless, the overall decision surface remains broadly similar to the previous two strategies, indicating a degree of structural robustness in the fitted tree.

Table 2 summarizes SVMODT performance across different maximum feature selection strategies compared to the Python STree benchmark. Overall, predictive accuracy is largely consistent across strategies, with differences generally within a few percentage points. For most datasets, all three maximum feature strategies; Constant, Decrease, and Random perform similarly to the STree benchmark, demonstrating the robustness of SVMODT to variations in maximum feature selection. Some datasets, such as Wine, Cardiotocography-10, and Dermatology, exhibit notable differences. For example, the Decrease strategy underperforms on Cardiotocography-10 relative to Constant and Random, while Random selection yields the highest accuracy for Fertility. In datasets with smaller feature sets or fewer classes, differences between strategies are minimal, indicating that the choice of maximum feature strategy has limited impact when feature dimensionality is low. These results suggest that SVMODT is generally stable across maximum feature selection strategies, but careful selection of the strategy may yield marginal gains on larger or more complex datasets.

Feature Selection Method At each node of the tree, a subset of candidate features is selected from the full feature set $\mathcal{F}$ to construct the SVM-based oblique split. This process encourages diversity across nodes in the tree. Let $p = |\mathcal{F}|$ denote the total number of available features. At depth d, the

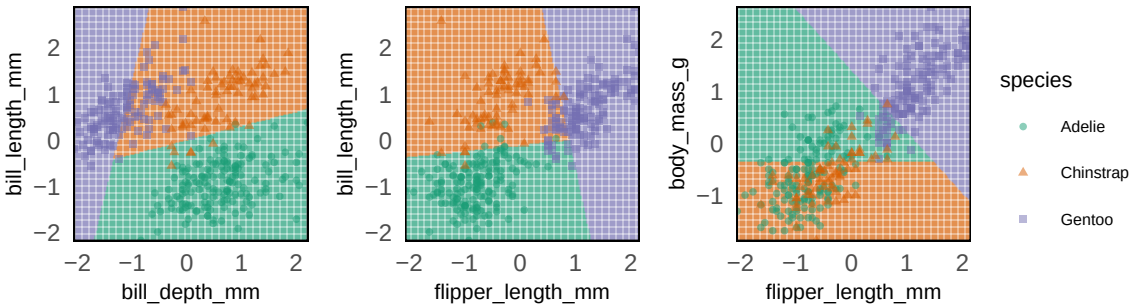


Figure 3: Comparison of random, mutual, and correlated feature selection methods using SVMODT on the Palmer Penguins dataset.

algorithm first determines the maximum number of features m_d that may be considered for splitting. This value is computed according to the selected maximum-feature strategy. A subset of up to m_d features is then chosen using a specified feature selection method. The package supports multiple feature selection approaches:

- **Random Selection:** A subset of features is sampled uniformly at random from the available feature set. This strategy introduces stochasticity into the tree construction process.
- **Mutual Information Ranking:** Features are ranked based on their mutual information with the target variable. Mutual information measures the statistical dependency between a feature and the class labels. Mutual information quantifies the reduction in uncertainty about the class label obtained by observing a given feature, thereby measuring the statistical dependency between predictors and the response. Features with higher mutual information are considered more informative for classification, and the top-ranked variables are selected for model training. This ranking is implemented using the **FSelectorRcpp** package.
- **Correlation-Based Filtering:** Features are ranked using a univariate association measure between each predictor and the response variable. Predictors with larger association values are considered more relevant, and the highest-ranked features are retained for model fitting.

Figure 3 illustrates the effect of alternative feature selection strategies on the splits generated by SVMODT. The resulting decision surfaces differ substantially across specifications, demonstrating the flexibility of the proposed framework and the sensitivity of tree structure to the feature selection mechanism employed at each node. Under the random strategy, the model selects `bill_depth_mm` and `bill_length_mm`, yielding boundaries based on random sampling without replacement from the full feature set. The mutual information strategy selects variables with the strongest dependence on the response, namely `flipper_length_mm` and `bill_length_mm`, producing partitions aligned with the most informative predictors. In contrast, the correlation-based strategy selects `flipper_length_mm` and `body_mass_g`, reflecting variables most strongly linearly associated with the class labels.

To further encourage diversity across tree nodes, the algorithm incorporates a feature reuse penalty mechanism. Previously selected features are tracked in the set $\mathcal{F}_{\text{used}}$, and a penalty parameter λ_{pen} reduces the probability that these features are repeatedly chosen at deeper levels of the tree. After the candidate feature subset $\mathcal{F}_d$ is selected, the corresponding predictors are standardized at the node level. Features that become constant after preprocessing are removed. If no valid features remain, the node is converted into a leaf node. The selected feature subset is then used to train the SVM model responsible for generating the oblique decision boundary at the node. For strategies involving random feature subsets, multiple candidate subsets may be evaluated. For each candidate subset, a Support Vector Machine is fitted and the resulting split is assessed using an impurity based criterion such as information gain. The subset that produces the highest impurity reduction is selected for the node.

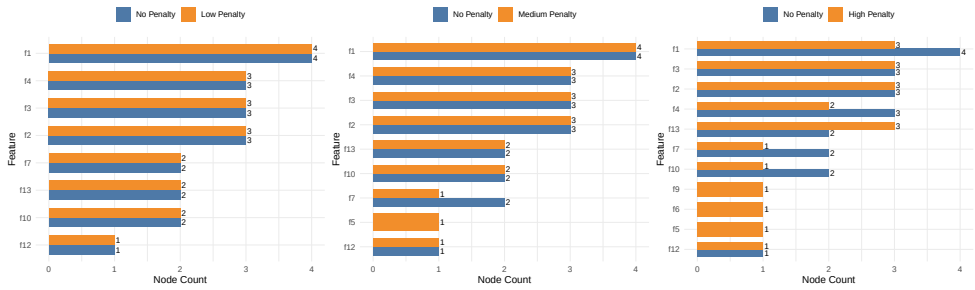
Table 3 compares classification performance of different feature selection strategies evaluated across ten benchmark datasets. Across the datasets, the results indicate that all feature selection strategies achieve competitive performance, with relatively small differences in accuracy. The SVM based oblique splitting mechanism remains robust to the specific feature selection approach used at each node. Random feature selection performs consistently well and achieves accuracy values that are comparable to, and occasionally exceed, the benchmark model. Mutual Information and Correlation based methods provide slightly more stable performance on certain datasets where informative predictors might exhibit stronger statistical relationships with the target variable.

Feature Penalty Mechanism To encourage feature diversity across tree nodes and reduce repeated reliance on the same predictors, SVMODT incorporates a feature penalisation mechanism during

Table 3: Comparison of mean prediction accuracy across feature selection strategies for SVMODT, benchmarked against the Python STree implementation. All SVMODT models were trained using $d_{\max} = 15$. The random strategy selects candidate predictors using random subset sampling $n_{\text{subsets}} = 10$; the mutual information strategy ranks predictors according to their dependency with the response variable; the correlation strategy ranks predictors using absolute correlation with the response. STree does not employ an explicit pre-split feature ranking mechanism, instead optimising the node classifier directly on the available predictors. Bold values indicate the highest prediction accuracy for each dataset.

Dataset	N	X	L	STree (Benchmark)	Random	Mutual Info	Correlation
WDBC Diagnosis	569	30	2	**0.970 ± 0.003**	0.967 ± 0.003	0.968 ± 0.006	**0.970 ± 0.004**
Iris	150	4	3	**0.965 ± 0.010**	0.955 ± 0.010	0.960 ± 0.009	0.961 ± 0.008
Echocardiogram	131	10	2	0.845 ± 0.008	0.848 ± 0.005	**0.851 ± 0.007**	0.847 ± 0.007
Fertility	100	9	2	0.874 ± 0.010	**0.880 ± 0.003**	0.878 ± 0.009	0.878 ± 0.009
Wine	178	13	3	0.959 ± 0.008	**0.962 ± 0.016**	**0.962 ± 0.016**	0.961 ± 0.012
Cardiotocography-3	2126	21	3	**0.903 ± 0.003**	0.900 ± 0.007	**0.903 ± 0.002**	0.899 ± 0.005
Cardiotocography-10	2126	21	10	0.795 ± 0.018	0.790 ± 0.008	**0.798 ± 0.006**	0.793 ± 0.007
Ionosphere	351	33	2	**0.892 ± 0.015**	0.889 ± 0.020	0.888 ± 0.010	0.881 ± 0.012
Dermatology	366	34	6	0.959 ± 0.004	0.958 ± 0.011	**0.969 ± 0.007**	0.966 ± 0.003

feature selection. Features that have already been selected in earlier splits receive a reduced relevance score when candidate variables are evaluated at deeper nodes. This encourages the tree to explore alternative predictors rather than repeatedly relying on a small subset of dominant variables.



(a) No Penalty **(b)** Low (0.2) **(c)** Medium (0.5)

Figure 4: Comparison of Low, Medium and High penalty on repeated features by SVMODT on Wine dataset.

Figure 4 illustrates the effect of varying the penalisation strength on the fitted SVMODT model. Separate trees are estimated under different penalty values, after which the number of distinct features used and their selection frequencies are recorded. As the penalty increases, the algorithm is increasingly encouraged to utilise a broader set of predictors, thereby promoting structural diversity within the tree.

Let $\mathcal{F}_{\text{used}}$ denote the set of features previously selected in earlier nodes. Suppose s_j represents the original relevance score of feature j, such as its mutual information with the response variable. The penalised score, denoted $\tilde{s}_j$, is then defined as

$$\tilde{s}_j = \begin{cases} s_j, & \text{if } j \notin \mathcal{F}_{\text{used}}, \\ (1 - \lambda_{\text{pen}}) s_j, & \text{if } j \in \mathcal{F}_{\text{used}}, \end{cases}$$

where $\lambda_{\text{pen}} \in [0, 1]$ controls the strength of the penalty. A value of $\lambda_{\text{pen}} = 0$ implies no penalisation, whereas larger values increasingly discourage the reuse of previously selected variables.

After selecting the feature subset $\mathcal{F}_d$ at depth d, the set of previously used features is updated according to

$$\mathcal{F}'_{\text{used}} = \mathcal{F}_{\text{used}} \cup \mathcal{F}_d.$$

Table 4 compares SVMODT performance under different feature penalisation strategies compared to the benchmark (None). Overall, the impact of penalising previously used features varies across datasets. On datasets such as WDBC Diagnosis, Wine, and Dermatology, applying low or medium feature penalisation yields slightly higher prediction accuracy than the unpenalised benchmark. For

Table 4: Comparison of mean prediction accuracy (mean ± standard deviation) across feature penalisation strategies for SVMODT. All models were trained using $d_{\max} = 15$, mutual-information feature ranking, and adaptive maximum feature selection. No penalisation corresponds to reuse of previously selected features being unrestricted. Low, medium, and high penalisation correspond to feature penalty weights of $\lambda = 0.2$, $\lambda = 0.5$, and $\lambda = 0.8$, respectively. Bold values indicate the highest accuracy for each dataset.

Dataset	N	X	L	None (Benchmark)	Low	Medium	High
WDBC Diagnosis	569	30	2	0.946 ± 0.005	**0.950 ± 0.004**	0.949 ± 0.007	0.949 ± 0.007
Iris	150	4	3	**0.966 ± 0.006**	**0.966 ± 0.006**	**0.966 ± 0.006**	**0.966 ± 0.006**
Echocardiogram	131	10	2	**0.855 ± 0.001**	**0.855 ± 0.001**	**0.855 ± 0.001**	**0.855 ± 0.001**
Fertility	100	9	2	**0.881 ± 0.000**	**0.881 ± 0.000**	**0.881 ± 0.000**	**0.881 ± 0.000**
Wine	178	13	3	0.950 ± 0.015	0.948 ± 0.016	**0.951 ± 0.013**	0.947 ± 0.011
Cardiotocography-3	2126	21	3	**0.883 ± 0.006**	**0.883 ± 0.004**	0.882 ± 0.003	0.880 ± 0.004
Cardiotocography-10	2126	21	10	**0.782 ± 0.007**	0.775 ± 0.008	0.776 ± 0.010	0.739 ± 0.034
Ionosphere	351	33	2	**0.869 ± 0.018**	0.868 ± 0.016	0.868 ± 0.017	0.868 ± 0.017
Dermatology	366	34	6	0.974 ± 0.006	0.977 ± 0.006	**0.978 ± 0.006**	0.968 ± 0.009

Table 5: Comparison of tree structure and predictive accuracy across minimum sample thresholds on the WDBC dataset. All SVMODT models were trained using the random maximum feature strategy and mutual information feature ranking. Higher minimum sample thresholds restrict further node splitting and therefore produce shallower tree structures.

Min Samples	Splits	Leaves	Total Nodes	Accuracy
10	3	4	7	0.9739
50	2	3	5	0.9913

example, Wine achieves the highest accuracy under Medium penalisation (0.951 ± 0.013). Several datasets, including Iris, Echocardiogram, and Fertility, show identical performance across all penalisation levels, suggesting that feature reuse does not influence accuracy in these cases. In datasets with more complex class structures or imbalanced classes, such as Cardiotocography-10, high feature penalisation reduces predictive performance (0.739 ± 0.034), indicating that excessive penalisation can hinder model learning. In summary, moderate feature penalisation can provide marginal improvements in predictive accuracy, while excessive penalisation may be detrimental, particularly for datasets with large numbers of classes or complex patterns.

Pruning Pruning in SVMODT operates at both the node level and the global tree level to control model complexity, reduce overfitting, and improve generalisation. Unlike conventional decision trees that rely solely on pruning after tree construction, SVMODT incorporates pre-split pruning mechanisms that govern whether additional partitions should be created during training.

A primary control is the minimum node size parameter, denoted min_n, which specifies the minimum number of observations required for a node to be eligible for further splitting. If the number of observations at node v falls below this threshold, the node is converted into a terminal leaf. This prevents unstable support vector machine fits in very small partitions and reduces the likelihood of learning decision boundaries driven by sampling noise. From Table 5, increase in the minimum sample threshold, min_n, produces shallower tree structures by restricting further splits in smaller nodes. This reduction in tree complexity can mitigate overfitting and, in several cases, improve out-of-sample predictive accuracy.

A second source of pruning is the maximum tree depth parameter, d_{max}, which limits the number of recursive splits from the root node to any terminal node. Shallow trees produce simpler and more interpretable decision rules, whereas deeper trees allow greater flexibility and more localised decision boundaries. Constraining depth therefore provides a direct mechanism for balancing interpretability against predictive complexity.

Table 6 compares tree structure and predictive accuracy across different values of the maximum tree depth parameter. Increasing `max_depth` permits additional recursive splits, resulting in larger trees with more leaves and internal nodes. Expanding the depth from 3 to 5 nearly doubles the number of splits and increases accuracy from 0.8826 to 0.8991, indicating that the shallower tree underfits the data. However, further increasing the depth from 5 to 7 produces only marginal structural growth, while predictive accuracy remains unchanged at 0.8991. A depth of 5 was sufficient to capture the underlying decision structure, and deeper trees provide no additional benefit. Consequently, d_{max} acts as an effective pruning parameter, where moderate values balance model flexibility and predictive performance.

Table 6: Comparison of tree structure and accuracy across different maximum tree depths on the Cardiotocography dataset.

Max Depth	Splits	Leaves	Total Nodes	Accuracy
3	6	7	13	0.8826
5	12	13	25	0.8991
7	14	15	29	0.8991

Table 7: Comparison of tree structure and accuracy across different minimum impurity threshold values on the Cardiotocography dataset.

Impurity Threshold	Splits	Leaves	Total Nodes	Accuracy
0.005	12	13	25	0.8991
0.01	10	11	21	0.8991
0.1	6	7	13	0.8826

A third mechanism is the minimum information gain threshold, implemented through an impurity improvement criterion. At each node, the candidate split induced by the support vector machine is accepted only if the resulting reduction in class impurity exceeds a predefined threshold (τ_I). If the improvement in impurity is negligible, the node is terminated rather than expanded further. This avoids unnecessary partitions that offer little predictive value and acts as a structural penalty on overly complex trees.

Table 7 reports the effect of varying the minimum impurity threshold on tree complexity and predictive accuracy for the Cardiotocography dataset. Increasing the threshold imposes a stricter requirement for accepting new splits, thereby reducing overall tree size. Raising the threshold from 0.005 to 0.01 decreases the number of splits from 12 to 10 and total nodes from 25 to 21, while accuracy remains unchanged at 0.8991. This indicates that several low-gain splits can be removed without loss of predictive performance. However, a substantially larger threshold of 0.1 further contracts the tree to only 6 splits and 13 total nodes, accompanied by a decline in accuracy to 0.8826. Excessive pruning prevents the model from capturing important class structure. Overall, the minimum impurity threshold serves as an effective pruning mechanism, where moderate values simplify the tree without sacrificing accuracy, whereas overly restrictive values may induce underfitting.

Together, min_n, d_{max} and τ_I the information gain threshold form a complementary pruning framework. The minimum node size controls local sample adequacy, maximum depth constrains global tree growth, and information gain ensures that each additional split contributes meaningful class separation. Their joint use enables SVMODT to maintain predictive flexibility while limiting overfitting, particularly in small, noisy, or high-dimensional datasets.

Class Weight Handling To address class imbalance, the algorithm allows three weighting schemes:

- **None:** All classes are assigned equal weight, such that $w_c = 1 \; \forall c \in C$
- **Balanced:** Class weights are inversely proportional to the class frequencies in the training data. The weight assigned to class c is given by $w_c = \frac{n}{K \cdot n_c}$, where n denotes the total number of training samples, K is the number of classes, and n_c is the number of samples belonging to class c.
- **Custom:** Users may specify their own class weights w_c for each class $c \in C$, allowing domain-specific adjustments based on prior knowledge or application requirements.

Figure 5 illustrates the effect of alternative class weighting schemes on the decision boundaries produced by SVMODT. In the first panel, equal weights are assigned to all classes. In the second panel, balanced weights are used, whereby class weights are set inversely proportional to the number of observations belonging to each class at the node. In the final panel, a custom weighting scheme is imposed for illustrative purposes, with weights specified as Adelie = 2, Gentoo = 1, and Chinstrap = 5.

Under balanced weighting, the resulting decision boundaries remain broadly similar to those obtained under equal weighting, although the separating regions shift modestly toward the centre of the feature space. This reflects the increased influence of relatively smaller classes during local SVM estimation. In contrast, the custom weighting specification produces substantially different decision regions. Owing to the large weight assigned to Chinstrap penguins, the model allocates a more expansive and pronounced classification region to that class. Adelie penguins form two detached decision regions, indicating a redistribution of boundary geometry in response to the weighting scheme. By comparison, the Gentoo decision region changes only marginally, consistent with its comparatively low assigned weight. These results demonstrate that node-specific class weighting

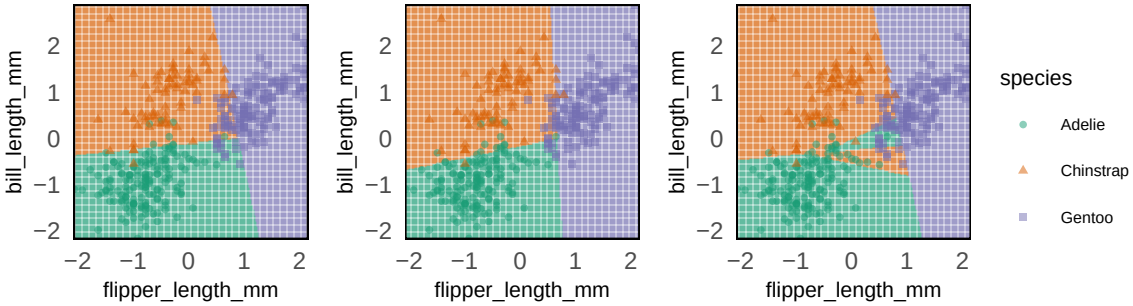


Figure 5: Comparison of equal, balanced, and custom weighting schemes using SVMODT on the Palmer Penguins dataset.

Table 8: Comparison of macro-averaged F1 score (mean ± sd) across class weighting strategies. All SVMODT models are trained with $d_{\max} = 15$, $\text{strat}_m =$ `"mutual"`. None: $\text{strat}_w =$ `"none"`; Balanced: $\text{strat}_w =$ `"balanced"`. Bold values denote the highest F1 score for each dataset.

Dataset	N	X	L	Class Imbalance Ratio	STree	None	Balanced
WDBC Diagnosis	569	30	2	0.5938375	**0.968 ± 0.004**	0.943 ± 0.008	0.948 ± 0.008
Iris	150	4	3	1.0000000	**0.965 ± 0.010**	0.963 ± 0.006	0.945 ± 0.015
Echocardiogram	131	10	2	0.4886364	0.792 ± 0.009	**0.800 ± 0.006**	0.752 ± 0.040
Fertility	100	9	2	0.1363636	0.466 ± 0.003	0.468 ± 0.001	**0.560 ± 0.041**
Wine	178	13	3	0.6760563	**0.959 ± 0.008**	0.954 ± 0.010	0.952 ± 0.017
Cardiotocography-3	2126	21	3	0.1063444	0.812 ± 0.007	0.766 ± 0.017	**0.827 ± 0.009**
Cardiotocography-10	2126	21	10	0.0915371	0.716 ± 0.011	0.586 ± 0.071	**0.746 ± 0.011**
Ionosphere	351	33	2	0.5600000	**0.879 ± 0.016**	0.855 ± 0.021	0.858 ± 0.020
Dermatology	366	34	6	0.1785714	0.946 ± 0.013	**0.956 ± 0.009**	0.950 ± 0.015

provides a flexible mechanism for altering local decision boundaries and may be particularly useful when class imbalance or asymmetric misclassification costs are present.

Table 8 presents the macro-averaged F1 scores obtained under different class weighting strategies across ten benchmark datasets. The results indicate that class weighting has a notable impact on predictive performance across datasets. For datasets exhibiting multiple classes or pronounced class imbalance, such as Cardiotocography and Fertility, the choice of weighting strategy substantially affects the macro-averaged F1 score. Specifically, the Balanced weighting scheme frequently improves performance relative to the unweighted (None) configuration. Conversely, in datasets with more balanced class distributions, differences between weighting strategies are smaller, although the highest F1 scores are often achieved with either the Balanced or Custom weights Overall, the results indicate that selecting an appropriate class weighting strategy is particularly important for datasets with multiple classes or significant class imbalance, while for more balanced datasets, weighting has a minimal impact.

Algorithm

Having outlined the principal components of SVMODT, we now present the complete training procedure. The preceding sections described each mechanism individually, including adaptive feature subsetting, feature selection strategies, pruning controls, feature penalisation, and class weighting. Algorithm 4 integrates these components into a unified recursive tree-building framework.

Algorithm 4: SVMODT: SVM-based Oblique Decision Tree with Enhanced Splitting

Input: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$: training dataset, d: current depth, $d_{\max}$: maximum depth, $n_{\min}$: minimum samples, m_{base}: base number of features, $\mathcal{C}$: set of all possible class labels, strat_m: feature selection strategy, strat_w: class weight strategy, $\text{method}_{\mathcal{F}}$: feature subset method, measure_I: impurity measure

Output: *tree*: leaf node or data split $\mathcal{D}'^{+}, \mathcal{D}'^{-}$

function SVMODT($\mathcal{D}, d, d_{\max}, n_{\min}, m_{base}, method_{\mathcal{F}}, measure_I, strat_m, strat_w, \mathcal{F}_{used}, \lambda_{pen}, \tau_I, \mathcal{C}$):

- $n \leftarrow |\mathcal{D}|$
- $\mathcal{Y} \leftarrow \{y_i\}_{i=1}^{n}$ // Extract labels
- // Stopping conditions
- **if** $d > d_{\max}$ ***or*** $|unique(\mathcal{Y})| = 1$ ***or*** $n < n_{\min}$ **then**
  - **return** create_leaf_node(mode($\mathcal{Y}$), $n, \mathcal{C}$)
- // Dynamic feature selection and scaling
- $m_d \leftarrow$ calculate_dynamic_max_features($d, m_{\text{base}}, \text{strat}_m$)
- $\mathcal{F}_d \leftarrow$ select_features_with_penalty($\mathcal{D}, m_d, \text{method}_{\mathcal{F}}, \mathcal{F}_{\text{used}}, \lambda_{\text{pen}}$)
- $\mathbf{X}_{\text{scaled}}, \text{scaler} \leftarrow$ scale_node($\mathcal{D}[\mathcal{F}_d]$)
- $\mathcal{F}_d \leftarrow$ features($\mathbf{X}_{\text{scaled}}$) // Update to non-constant features
- **if** $|\mathcal{F}_d| = 0$ **then**
  - **return** create_leaf_node(mode($\mathcal{Y}_{\surd}$), $n, \mathcal{C}$) // No valid features
- $k \leftarrow |\text{unique}(\mathcal{Y}_{\surd})|$ // Number of classes at node
- $I_{\text{p}} \leftarrow I(\mathcal{Y}, \text{measure}_I)$ // Parent impurity
- // Binary vs Multiclass handling
- **if** $k = 2$ **then**
  - (best_model, $\mathcal{I}_L, \mathcal{I}_R$) $\leftarrow$ BinarySplit($\mathbf{X}_{scaled}, \mathcal{Y}, strat_w$) // Refer to Algorithm 2
- **else**
  - (best_model, $\mathcal{I}_L, \mathcal{I}_R, c_{\text{split}}$) $\leftarrow$ MulticlassSplit($\mathbf{X}_{scaled}, \mathcal{Y}, strat_w, I_{parent}, measure_I, \tau_I, n$) // Refer to Algorithm 3
- **if** *best_model* $= null$ ***or*** $|\mathcal{I}_L| = 0$ ***or*** $|\mathcal{I}_R| = 0$ **then**
  - **return** create_leaf_node(mode($\mathcal{Y}$), $n, \mathcal{C}$)
- // Handle small children
- **if** $|\mathcal{I}_L| < n_{\min}$ ***and*** $|\mathcal{I}_R| < n_{\min}$ **then**
  - **return** create_leaf_node(mode($\mathcal{Y}_{\surd}$), $n, \mathcal{C}$) // Both children too small
- // Update used features for penalty mechanism
- $\mathcal{F}'_{\text{used}} \leftarrow \mathcal{F}_{\text{used}} \cup \mathcal{F}_d$
- // Create internal node
- *node* $\leftarrow$ create_internal_node(best_model, $\mathcal{F}_d$, scaler, d, n)
- // Recursively build children
- **if** $|\mathcal{I}_L| \geq n_{\min}$ **then**
  - *node*.left $\leftarrow$ SVMODT($\mathcal{D}[\mathcal{I}_L], d+1, d_{\max}, n_{\min}, m_{\text{base}}, \ldots, \mathcal{F}'_{\text{used}}, \ldots, \mathcal{C}$)
- **else**
  - *node*.left $\leftarrow$ create_leaf_node(mode($\mathcal{Y}[\mathcal{I}_L]$), $|\mathcal{I}_L|, \mathcal{C}$)
- **if** $|\mathcal{I}_R| \geq n_{\min}$ **then**
  - *node*.right $\leftarrow$ SVMODT($\mathcal{D}[\mathcal{I}_R], d+1, d_{\max}, n_{\min}, m_{\text{base}}, \ldots, \mathcal{F}'_{\text{used}}, \ldots, \mathcal{C}$)
- **else**
  - *node*.right $\leftarrow$ create_leaf_node(mode($\mathcal{Y}[\mathcal{I}_R]$), $|\mathcal{I}_R|, \mathcal{C}$)
- **return** *node*

4 Experiments

We evaluate the predictive performance of SVMODT against a range of competing decision tree and support vector machine-based classification methods. These benchmarks include Classification and Regression Trees (CART) using **rpart**, linear support vector machines using **e1071**, logistic regression using **nnet**, STree, and Accelerated Oblique Random Survival Forests using **aorsf**. Collectively, these comparators provide a broad reference set spanning conventional axis-parallel trees, linear margin-based classifiers, and alternative oblique tree-based approaches.

To optimise the predictive performance of SVMODT, we first conducted a cross-validated grid

search over a predefined hyperparameter space for each benchmark dataset. Model selection was based on repeated 5-fold cross-validation across three independent random seeds. For each candidate configuration, the mean classification accuracy and its standard deviation across seeds were recorded. The final tuning configuration for each dataset was selected as the model with the highest mean accuracy, with ties resolved in favour of the lowest cross-seed standard deviation to prioritise stability. The selected hyperparameters were then re-evaluated using the remaining seeds under an identical 5-fold cross-validation design to obtain the reported out-of-sample performance of **SVMODT***.

The grid search tuned the principal structural and feature-selection components of SVMODT. These included: (i) the feature selection method (`random`, `mutual`, `cor`), corresponding respectively to random feature ranking, mutual information, and correlation-based screening; (ii) the class weighting scheme (`none`, `balanced`); (iii) the maximum feature strategy (`constant`, `decrease`, `random`), which determines how many predictors are available at each node; (iv) whether to apply feature penalisation (`TRUE`, `FALSE`); (v) the feature penalty weight $(0.3, 0.6)$ when penalisation was active; (vi) the minimum impurity decrease threshold $(0.001, 0.01, 0.1)$, used as a stopping and pruning criterion; and (vii) the node-level maximum number of candidate features, chosen adaptively as either $\lfloor \sqrt{p} \rfloor$ or p, where p is the total number of predictors. For random feature strategies, the candidate proportion was sampled uniformly from the interval $[0.1, 0.9]$.

STree* denotes the Python implementation of STree using the dataset-specific kernel and regularisation parameters reported by the original authors (see Section 4.2; Montañana et al. (2025)). To ensure fair benchmarking, the CART baselines implemented via `rpart` were also tuned using cross-validated grid search over tree depth and complexity penalty under the same resampling framework. Table 9 and 10 presents mean prediction accuracy across seven classifiers on nine benchmark datasets.

Overall competitiveness

SVMODT* remains strongly competitive across the benchmark suite, achieving the highest standalone accuracy on **Echocardiogram** (0.854) and **Dermatology** (97.9) while matching the best result on **Iris** (0.971) and **Fertility**. It also performs well on several other datasets, including **Wine** (0.979), **WDBC** (0.964), and **Cardiotocography-10** (0.795). SVMODT* relies exclusively on linear support vector machine splits at each node, whereas several competing methods including STree* and AORSF benefit from more flexible non-linear or ensemble-based decision boundaries.

Performance relative to STree*

Despite this structural simplicity, SVMODT* remains competitive with STree* across most datasets. STree* attains the strongest results on **WDBC** (0.973), **Ionosphere** (0.947), and several multiclass tasks, but these gains often arise when non-linear kernels such as radial basis function (RBF) or polynomial kernels are used. By contrast, SVMODT* uses only linear node-wise hyperplanes, making the resulting tree substantially easier to interpret and requiring fewer hyperparameters to tune. On datasets where STree* uses a linear kernel, such as **Iris**, **Wine**, and **Dermatology**, the performance gap narrows considerably, and SVMODT* either matches or remains within a small margin of STree*.

Performance relative to default SVMODT

Hyperparameter optimisation produces modest but meaningful gains on selected datasets. Improvements are observed on **Iris** (+0.005), **Dermatology** (+0.007), **Echocardiogram** (+0.008), and **Cardiotocography-10** (+0.003) relative to the default specification. However, tuning does not uniformly improve performance: the default SVMODT remains stronger on **Wine** (+0.008) and **Fertility** (+0.007). This indicates that the untuned configuration already provides a robust baseline, while more aggressive optimisation may overfit smaller or low-variance datasets. Accordingly, SVMODT appears relatively stable even without extensive parameter search.

Performance relative to tree-based baselines

Both SVMODT variants generally outperform conventional single-tree CART models on most datasets. For example, on **WDBC**, **Wine**, **Iris**, and **Dermatology**, SVMODT exceeds axis-aligned CART accuracy. However, on **Cardiotocography-3** and **Cardiotocography-10**, tuned CART remains highly competitive, suggesting that deep axis-parallel trees can still perform strongly when class structure aligns well with recursive univariate splits. Relative to ensemble tree methods, **AORSF** is the strongest overall competitor, achieving the best results on **Wine** (0.988) while remaining highly competitive elsewhere. This is expected given the variance-reduction benefits of ensemble learning. Nevertheless, SVMODT

Table 9: Comparison of Mean Prediction Accuracy (mean ± sd) across classifiers (Part 1). Bold denotes the highest accuracy per dataset.

Dataset	N	X	L	CART*	SVM (linear)	AORSF	Log. Reg.
WDBC Diagnosis	569	30	2	0.925 ± 0.006	0.970 ± 0.004	0.971 ± 0.004	0.946 ± 0.007
Iris	150	4	3	0.931 ± 0.008	0.966 ± 0.011	0.955 ± 0.004	0.961 ± 0.011
Echocardiogram	131	10	2	0.853 ± 0.005	0.846 ± 0.004	0.809 ± 0.014	0.791 ± 0.018
Fertility	100	9	2	**0.881 ± 0.000**	0.874 ± 0.010	**0.881 ± 0.000**	0.840 ± 0.014
Wine	178	12	3	0.864 ± 0.025	0.961 ± 0.005	**0.988 ± 0.002**	0.983 ± 0.005
Cardiotography-3	2126	21	3	**0.919 ± 0.003**	0.897 ± 0.002	0.911 ± 0.002	0.892 ± 0.004
Cardiotography-10	2126	21	10	0.819 ± 0.006	0.820 ± 0.005	0.807 ± 0.003	**0.821 ± 0.005**
Ionosphere	351	33	2	0.887 ± 0.013	0.879 ± 0.005	0.921 ± 0.005	0.877 ± 0.012
Dermatology	366	34	6	0.929 ± 0.007	0.968 ± 0.006	0.976 ± 0.005	0.968 ± 0.010

Table 10: Comparison of Mean Prediction Accuracy (mean ± sd) across classifiers (Part 2). *STree** denotes Python STree with dataset-specific hyperparameters. *SVMODT** denotes SVMODT with tuned hyperparameters from grid search. Bold denotes the highest accuracy per dataset.

Dataset	N	X	L	PPTree	STree*	SVMODT (Default)	SVMODT*
WDBC Diagnosis	569	30	2	0.966 ± 0.006	**0.973 ± 0.004**	0.969 ± 0.003	0.964 ± 0.006
Iris	150	4	3	**0.971 ± 0.005**	0.966 ± 0.011	0.966 ± 0.006	**0.971 ± 0.006**
Echocardiogram	131	10	2	0.831 ± 0.011	0.848 ± 0.016	0.846 ± 0.006	**0.854 ± 0.003**
Fertility	100	9	2	**0.881 ± 0.000**	**0.881 ± 0.000**	0.874 ± 0.010	**0.881 ± 0.000**
Wine	178	12	3	0.961 ± 0.014	0.964 ± 0.010	0.978 ± 0.008	0.970 ± 0.008
Cardiotography-3	2126	21	3	0.889 ± 0.003	0.903 ± 0.003	0.902 ± 0.004	0.911 ± 0.006
Cardiotography-10	2126	21	10	0.763 ± 0.006	0.808 ± 0.007	0.792 ± 0.004	0.796 ± 0.005
Ionosphere	351	33	2	0.859 ± 0.014	**0.947 ± 0.006**	0.896 ± 0.011	0.886 ± 0.012
Dermatology	366	34	6	0.905 ± 0.025	0.968 ± 0.006	0.972 ± 0.006	**0.979 ± 0.004**

offers a substantially more interpretable single-tree alternative while retaining competitive predictive performance.

Summary

Overall, the results demonstrate that SVMODT bridges the gap between interpretability and predictive accuracy. It consistently outperforms conventional single decision trees, approaches the performance of more complex SVM-tree hybrids such as STree, and remains competitive with ensemble oblique forests on several datasets, all while preserving a transparent linear-split tree structure. SVMODT* demonstrates competitive predictive performance across the benchmark datasets, achieving the highest accuracy on several problems despite relying solely on linear node-wise hyperplanes. Hyperparameter optimisation yields modest but meaningful improvements on selected datasets, although the default configuration already serves as a strong and robust baseline, particularly for smaller or lower-dimensional problems where extensive tuning may overfit.

The feature penalisation mechanism, while producing modest improvements on average, demonstrates a clear structural effect: increasing λ_{pen} redistributes feature usage across nodes, promoting diversity in the learned decision boundaries. This behaviour mirrors the role of the `mtry` parameter in random forests, offering a single-tree analogue of the subspace-sampling heuristic that underpins ensemble diversity (Breiman et al., 1984). The adaptive maximum-feature strategies provide complementary control: the constant strategy delivers predictable computational cost, the decreasing strategy progressively reduces the number of candidate predictors available at deeper nodes, and the random strategy introduces stochasticity that can reduce overfitting on smaller datasets. The benchmark results also reveal a consistent pattern across pruning parameters. Moderate values of $n_{\min}$, $d_{\max}$, and τ_I reliably improve or maintain out-of-sample accuracy relative to their extremes, and their effects are largely orthogonal: minimum node size governs local sample adequacy, maximum depth constrains global tree growth, and the impurity threshold controls split quality. Practitioners can therefore treat them as independent controls, tuning each in turn under a standard cross-validation protocol. Overall, the results suggest that oblique decision trees combining adaptive feature selection, feature diversity mechanisms, and controlled pruning are especially effective for multiclass and high-dimensional classification tasks.

5 Conclusion

Across nine benchmark datasets, **SVMODT** with tuned hyperparameters (SVMODT*) consistently outperformed conventional axis-aligned CART and remained competitive with the more flexible STree* implementation despite relying exclusively on linear node-wise hyperplanes. On Echocardiogram (0.854), Dermatology (0.979), Iris (0.971), and Fertility, it achieved the strongest or joint-strongest

predictive performance among all methods considered. The untuned default configuration served as a robust baseline throughout, indicating that the proposed extensions deliver structural benefits independent of careful hyperparameter search.

This paper makes two concrete contributions. First, **STreeR** provides the first fully native R implementation of the **STree** algorithm, reconstructed without Python dependencies and validated against the original implementation across the full benchmark suite. Second, **SVMODT** extends STree through five mechanisms: adaptive feature subsetting, embedded feature selection via mutual information or correlation ranking, a feature penalisation scheme that discourages repeated predictor reuse, node-specific class weighting, and a three-parameter pruning framework, each of which can be tuned independently under standard cross-validation.

Three limitations bound the current work. Hyperparameter optimisation is computationally demanding because tree construction requires solving node-level SVM problems across many candidate configurations, a cost that grows with dataset size. The restriction to linear kernels prioritises interpretability but may limit performance when class boundaries are strongly nonlinear; the 6-point accuracy gap on Ionosphere relative to STree*, which uses an RBF kernel on that dataset, is consistent with this limitation. The evaluation is also restricted to moderate-sized tabular datasets; behaviour on large-scale, sparse, or ultra-high-dimensional problems remains untested.

These limitations motivate several directions for future development. A natural extension is an ensemble variant of SVMODT analogous to Oblique Decision Tree Ensemble (ODTE) (Montañana et al., 2025), incorporating the feature selection, penalisation, and class-weighting mechanisms introduced here. Such an approach could leverage bagging or boosting to reduce prediction variance while retaining interpretable node-level decision rules and feature importance summaries. Additional developments include support for nonlinear kernels, integration with scalable optimisation routines, parallelised model fitting, and extensions to regression settings. The package is available on Github and under active development.

6 Acknowledgements

The authors would like to thank Professor **Natalia Da Silva** and Professor **Dianne Helen Cook** for their constructive discussions and valuable insights regarding oblique decision trees and their implementations in R. The authors also acknowledge the use of artificial intelligence–assisted tools during the preparation of this manuscript. OpenAI's GPT models (OpenAI) were used for technical writing assistance, including improving clarity of exposition, refining document structure, and enhancing stylistic consistency. Anthropic's Claude models (Anthropic, 2026) were also used for coding refinement and implementation support, including assistance with code organisation, debugging, and software development workflows. All conceptual development, methodological design, data processing, experimental analysis, implementation decisions, and interpretation of results were conducted independently by the author. These tools were used solely as supportive aids and did not contribute to the scientific content, conclusions, or intellectual contributions of the work.

7 Appendix

Stree Algorithm

The following section describes the STree algorithm proposed by Montañana et al. (2021).

Algorithm 5: STree (Multi-class SVM-based oblique decision tree)

Input: $\mathcal{D}' = \{(x_i, y_i)\}_{i=1}^{l'}$: Data
Output: *tree*: the root node of an SVM-based oblique decision tree

function STree($\mathcal{D}'$):
  if *stopping_condition*$(\{y_i\}_{i=1}^{t})$ **then**
    return *create_leaf_node*$(mode(\{y_i\}_{i=1}^{t}))$ // Leaf node
  $k' \leftarrow num\_different\_labels(\{y_i\}_{i=1}^{t})$
    $r \leftarrow \frac{k'(k'-1)}{2}$
    $\mathcal{Y}' \leftarrow \{y'_1, \ldots, y'_{k'}\}$ // set of labels
  $I_{all} \leftarrow I(\{y_i\}_{i=1}^{t})$ // $I(\cdot)$ is an information theory measure
  if (OvO) **then**
    $n_{models} \leftarrow r$
  else
    $n_{models} \leftarrow k'$ // OvR
  for $j = 1$ **to** n_{models} **do**
    if $(j = k' = 2)$ **then**
      break // Two labels, one SVM is enough
    if (OvO) **then**
      Let c_a and c_b the pair of labels corresponding to index j
      $models[j] \leftarrow \text{SVM}(\mathcal{D}'^{\downarrow c_a} \cup \mathcal{D}'^{\downarrow c_b})$
    else
      $models[j] \leftarrow \text{SVM}(\mathcal{D}')$ using binary class y'_j (+) vs rest (–) // OvR
  $\mathcal{D}'^{+}, \mathcal{D}'^{-} \leftarrow models[j](\overrightarrow{x}_j) \quad \forall(\overrightarrow{x}) \in \mathcal{D}'$
  $I_j \leftarrow \frac{|\mathcal{D}'^{+}|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}'^{+}|}) + \frac{|\mathcal{D}'^{-}|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}'^{-}|})$
  $IG_j \leftarrow I_{all} - I_j$
  $b^* = \arg\max_{j=1,\ldots,n_{models}} IG_j$
  if $IG_{b^*} > 0$ **then**
    $node \leftarrow create\_node(models[b^*])$
    $node.left \leftarrow STree(\mathcal{D}'^{+})$
    $node.right \leftarrow STree(\mathcal{D}'^{-})$
  else
    return *create_leaf_node*$(model(\{y_i\}_{i=1}^{t}))$ // No split gain
  return *node*

SVMODT* Hyperparameters

Table 11 presents the hyperparameters selected through the grid search procedure.

Table 11: Hyperparameters Selected through the Grid Search Procedure.

Dataset	Max Depth	Feature Selection Method	Class Weight Strategy	Max Feature Strategy	Penalize Used Features	Feature Penalty Weight	Impurity Decrease
wdbc	10	mutual	none	random	FALSE	0.3	0.100
iris	10	cor	none	decrease	FALSE	0.3	0.001
echocardiogram	10	cor	none	constant	FALSE	0.3	0.001
fertility	10	cor	none	constant	FALSE	0.3	0.001
wine	10	cor	none	constant	FALSE	0.3	0.001
ctg3	10	mutual	balanced	decrease	TRUE	0.3	0.001
ctg10	10	mutual	balanced	random	TRUE	0.3	0.001
ionosphere	10	mutual	none	random	TRUE	0.3	0.001
dermatology	10	cor	none	decrease	FALSE	0.3	0.001

Bibliography

Anthropic. Claude 3.5 sonnet. Large Language Model, 2026. URL https://claude.ai. [p18]

M. Bala and R. Agrawal. Optimal decision tree based multi-class support vector machine. *Informatica*, 35(2), 2011. [p2, 4]

K. Bennett and J. Blue. A support vector machine approach to decision trees. In *1998 IEEE International Joint Conference on Neural Networks Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98CH36227)*, volume 3, pages 2396–2401 vol.3, 1998. doi: 10.1109/IJCNN.1998.687237. [p2, 4]

L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees.* Wadsworth, 1984. URL http://lyle.smu.edu/~mhd/8331f06/cart.pdf. [p1, 2, 17]

C. E. Brodley and P. E. Utgoff. Multivariate decision trees. *Machine Learning*, 19:45–77, 1995. URL https://api.semanticscholar.org/CorpusID:16836572. [p2, 3]

S. Bulò and P. Kontschieder. Neural decision forests for semantic image labelling. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 81–88, 2014. doi: 10.1109/CVPR.2014.18. [p3]

L. Cañete, R. Monroy, and M. Medina-Pérez. A review and experimental comparison of multivariate decision trees. *IEEE Access*, PP:1–1, 08 2021. doi: 10.1109/ACCESS.2021.3102239. [p1]

J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408:189–215, 2020. ISSN 0925-2312. doi: https://doi.org/10.1016/j.neucom.2019.10.118. URL https://www.sciencedirect.com/science/article/pii/S0925231220307153. [p4]

M. Chabbouh, S. Bechikh, E. Mezura-Montes, and L. Ben Said. Evolutionary optimization of the area under precision-recall curve for classifying imbalanced multi-class data. *Journal of Heuristics*, 31, 01 2025. doi: 10.1007/s10732-024-09544-z. [p3]

C. Cortes and V. Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995. [p2, 3]

N. Cristianini. An introduction to support vector machines and other kernel-based learning methods, 2000. [p2, 3]

N. da Silva, D. Cook, and E.-K. Lee. A projection pursuit forest algorithm for supervised classification. *Journal of Computational and Graphical Statistics*, 30(4):1168–1180, 2021. doi: 10.1080/10618600.2020.1870480. URL https://doi.org/10.1080/10618600.2020.1870480. [p3]

J.-x. Dong, A. Krzyzak, and C. Y. Suen. Fast svm training algorithm with decomposition on very large data sets. *IEEE transactions on pattern analysis and machine intelligence*, 27(4):603–618, 2005. [p4]

M. R. Frean. *Small nets and short paths: Optimising neural computation*. PhD thesis, 1990. [p3]

M. Friedl and C. Brodley. Decision tree classification of land cover from remotely sensed data. *Remote Sensing of Environment*, 61(3):399–409, 1997. ISSN 0034-4257. doi: https://doi.org/10.1016/S0034-4257(97)00049-7. URL https://www.sciencedirect.com/science/article/pii/S0034425797000497. [p3]

J. H. Friedman and J. W. Tukey. A projection pursuit algorithm for exploratory data analysis. *IEEE Transactions on computers*, 100(9):881–890, 2006. [p3]

S. I. Gallant. Optimal linear discriminants. *Eighth International Conference on Pattern Recognition*, pages 849–852, 1986. URL https://cir.nii.ac.jp/crid/1573668924476144256. [p3]

M. Ganaie, M. Tanveer, P. Suganthan, and V. Snasel. Oblique and rotation double random forest. *Neural Networks*, 153:496–517, 2022. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2022.06.012. URL https://www.sciencedirect.com/science/article/pii/S0893608022002258. [p2]

C.-W. Hsu and C.-J. Lin. A comparison of methods for multiclass support vector machines. *IEEE transactions on Neural Networks*, 13(2):415–425, 2002. [p4]

J. Kittler. Feature selection and extraction. *Handbook of Pattern Recognition and Image Processing*, pages 59–83, 1986. URL https://cir.nii.ac.jp/crid/1573950400671608448. [p3]

M. Koziol and M. Wozniak. *Multivariate Decision Trees vs. Univariate Ones*, pages 275–284. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. ISBN 978-3-540-93905-4. doi: 10.1007/978-3-540-93905-4_33. URL https://doi.org/10.1007/978-3-540-93905-4_33. [p3]

J. B. Kruskal. Toward a practical method which helps uncover the structure of a set of multivariate observations by finding the linear transformation which optimizes a new "index of condensation". In *Statistical computation*, pages 427–440. Elsevier, 1969. [p3]

Y. Lee, D. Cook, J. Park, and E.-K. Lee. Pptree: Projection pursuit classification tree. *Electronic Journal of Statistics*, 7(1):1369 – 1386, 2013. ISSN 1935-7524. doi: 10.1214/13-EJS810. [p3]

T. D. Lemmond, B. Y. Chen, A. O. Hatch, and W. G. Hanley. *An Extended Study of the Discriminant Random Forest*, pages 123–146. Springer US, Boston, MA, 2010. ISBN 978-1-4419-1280-0. doi: 10.1007/978-1-4419-1280-0_6. URL https://doi.org/10.1007/978-1-4419-1280-0_6. [p3]

H. Liu and R. Setiono. Feature transformation and multivariate decision tree induction. In S. Arikawa and H. Motoda, editors, *Discovey Science*, pages 279–291, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. ISBN 978-3-540-49292-4. [p3]

V. Menkovski, I. T. Christou, and S. Efremidis. Oblique decision trees using embedded support vector machines in classifier ensembles. In *2008 7th IEEE International Conference on Cybernetic Intelligent Systems*, pages 1–6, 2008. doi: 10.1109/UKRICIS.2008.4798937. [p2]

B. H. Menze, B. M. Kelm, D. N. Splitthoff, U. Koethe, and F. A. Hamprecht. On oblique random forests. In D. Gunopulos, T. Hofmann, D. Malerba, and M. Vazirgiannis, editors, *Machine Learning and Knowledge Discovery in Databases*, pages 453–469, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-23783-6. [p3]

J. Mingers. An empirical comparison of pruning methods for decision tree induction. *Machine Learning*, 4:227–243, 01 1989. doi: 10.1023/A:1022604100933. [p3]

R. Montañana, J. A. Gámez, and J. M. Puerta. Stree: A single multi-class oblique decision tree based on support vector machines. In E. Alba, G. Luque, F. Chicano, C. Cotta, D. Camacho, M. Ojeda-Aciego, S. Montes, A. Troncoso, J. Riquelme, and R. Gil-Merino, editors, *Advances in Artificial Intelligence*, pages 54–64, Cham, 2021. Springer International Publishing. ISBN 978-3-030-85713-4. [p4]

R. Montañana, J. A. Gámez, and J. M. Puerta. Stree: a single multi-class oblique decision tree based on support vector machines, 2021. [p2, 4, 18]

R. Montañana, J. A. Gámez, and J. M. Puerta. Odte—an ensemble of multi-class svm-based oblique decision trees. *Expert Systems with Applications*, 273:126833, 2025. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2025.126833. URL https://www.sciencedirect.com/science/article/pii/S0957417425004555. [p16, 18]

S. K. Murthy, S. Kasif, and S. Salzberg. A system for induction of oblique decision trees. *CoRR*, abs/cs/9408103, 1994. URL http://arxiv.org/abs/cs/9408103. [p1]

M. Nanda, K. Seminar, D. Nandika, and A. Maddu. A comparison study of kernel functions in the support vector machine and a comparison study of kernel functions in the support vector machine and its application for termite detection, 2018. [p4]

OpenAI. Chatgpt (gpt-4 version). URL https://chat.openai.com. Large language model. [p18]

G. Pagallo and D. Haussler. Boolean feature discovery in empirical learning. *Machine Learning*, 5:71–99, 1990. URL https://api.semanticscholar.org/CorpusID:5661437. [p1, 3]

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. [p2]

J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1:81–106, 1986. URL https://api.semanticscholar.org/CorpusID:189902138. [p3]

J. R. Quinlan. *C4. 5: programs for machine learning*. Elsevier, 2014. [p1, 3]

E. Reynolds, B. Callaghan, and M. Banerjee. Svm–cart for disease classification. *Journal of Applied Statistics*, 46(16):2987–3007, 2019. doi: 10.1080/02664763.2019.1625876. URL https://doi.org/10.1080/02664763.2019.1625876. PMID: 33012942. [p4]

C. Schaffer. Deconstructing the digit recognition problem. In D. Sleeman and P. Edwards, editors, *Machine Learning Proceedings 1992*, pages 394–399. Morgan Kaufmann, San Francisco (CA), 1992. ISBN 978-1-55860-247-2. doi: https://doi.org/10.1016/B978-1-55860-247-2.50056-5. URL https://www.sciencedirect.com/science/article/pii/B9781558602472500565. [p3]

F. Takahashi and S. Abe. Decision-tree-based multiclass support vector machines. volume 3, pages 1418 – 1422 vol.3, 12 2002. ISBN 981-04-7524-1. doi: 10.1109/ICONIP.2002.1202854. [p2, 4]

A. K. Y. Truong. Fast growing and interpretable oblique trees via logistic regression models. 2009. URL https://api.semanticscholar.org/CorpusID:57884720. [p3]

P. C. Young. Recursive estimation and time-series analysis: An introduction. 1984. URL https://api.semanticscholar.org/CorpusID:60181335. [p3]

Aneesh Agarwal
Monash University

aaga0022@student.monash.edu

Jack Jewson
Monash University
Department of Econometrics and Business Statistics, Monash University, Australia
Jack.Jewson@monash.edu

Erik Sverdrup
Monash University
Department of Econometrics and Business Statistics, Monash University, Australia
Erik.Sverdrup@monash.edu